

Workshop 2: A Practical Introduction to Quarto, GitHub, and Personal Websites (Using the Terminal)

Red Rock Data Science Conference 2026

GitHub Page: www.github.com/rbrown53/RRDS26

Rick Brown

Southern Utah University

2026-05-19

- 1 Installation Details
- 2 Quarto
- 3 Introduction to Unix
- 4 Unix Organization and Commands
- 5 Git and GitHub

Section 1

Installation Details

Important installations

You will need to install the following:

- **R** (<https://cloud.r-project.org>) or Python (<https://www.python.org/downloads>).
- RStudio (<https://posit.co/download/rstudio-desktop>) or Positron (<https://positron.posit.co/download.html>).
- GitHub Desktop Application (<https://desktop.github.com/download>).
- Git (<https://git-scm.com/install/>). If on Windows, that will also install Git Bash.

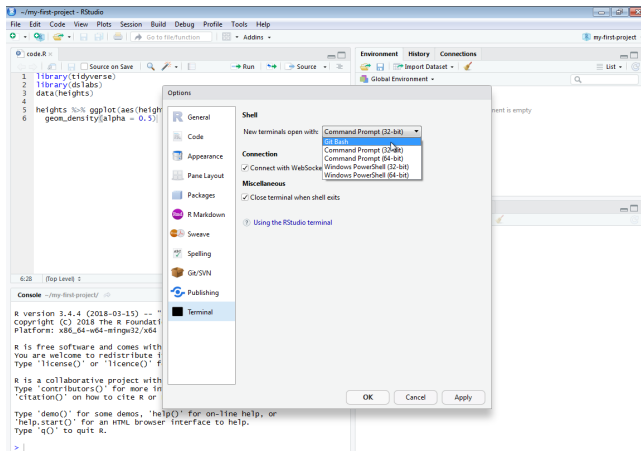
Note: When installing on Windows, make sure to chose the option to call the default branch “main” so it aligns better with GitHub. If you forget to do this, you can always change the default branch by typing

```
git config --global init.defaultBranch main
```

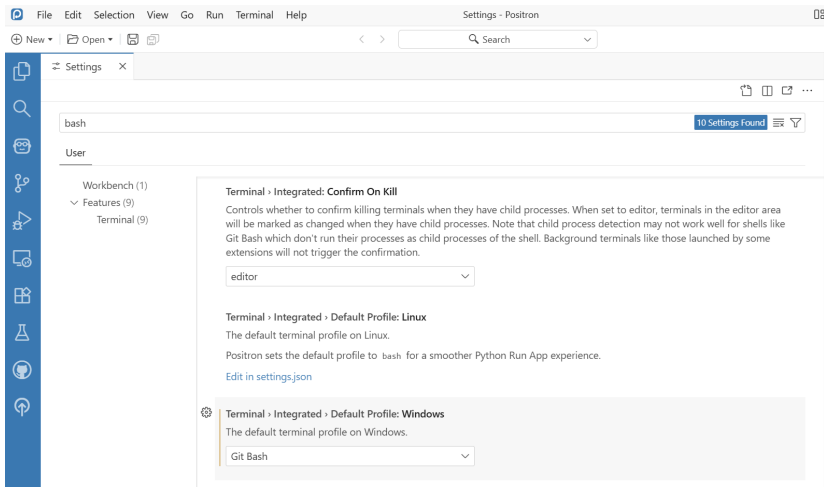
in the Git Bash terminal after installation.

For Windows: link Git Bash and RStudio

We can access the terminal either through RStudio or by opening Git Bash directly. For RStudio, set Git Bash as the default Unix shell: go to Tools then Global Options. . . , then select Terminal, then select Git Bash. It might have this already selected by default.



For Windows: link Git Bash and Positron



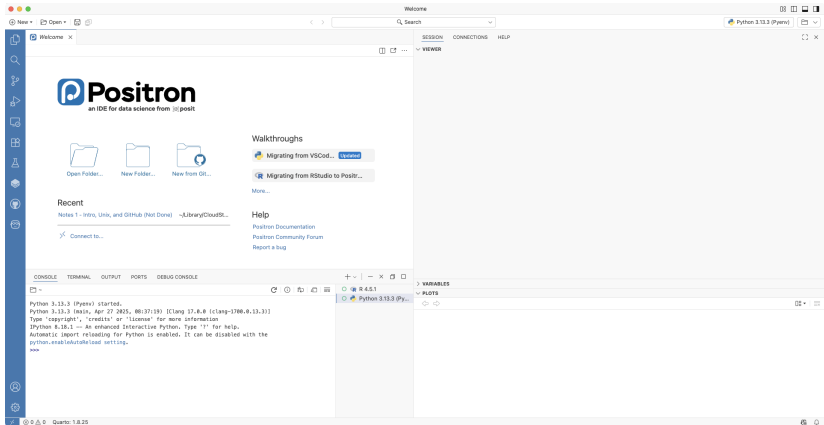
Quick Aside: Positron

Positron is a next-generation IDE (Integrated Development Environment) that is designed specifically for data science. It is like a mix between VS Code and RStudio and runs both **R** and Python (among other languages) natively.

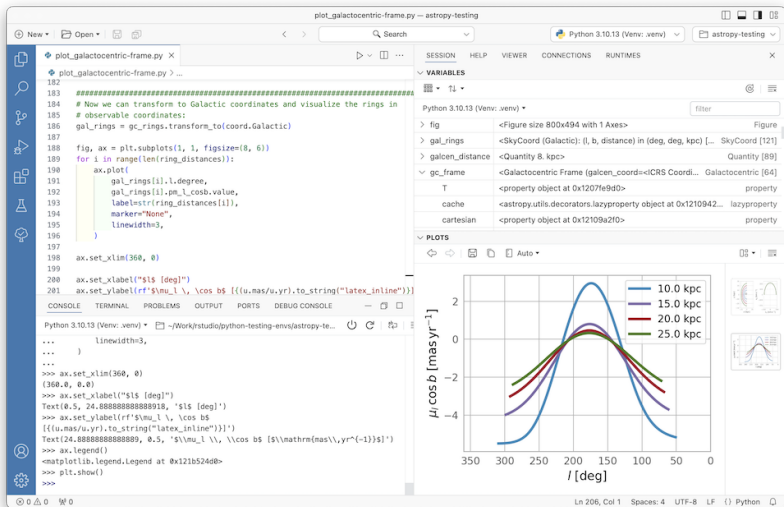
It is developed by Posit, which also created RStudio.

In this class, you can use RStudio or Positron. If you consider yourself primarily a Python programmer, then Positron may be a better fit for you than RStudio. However, I will give instructions in subsequent notes in RStudio, not Positron.

Quick Aside: Positron First Impressions



Quick Aside: Positron Typical Workflow



Quick Aside: Getting Started with Positron

When you first open Positron, it will want you to update the extensions. Click on the Extensions pane on the left side (the one with 4 squares and a red number on it). Then choose to update all extensions.

We will also need to make sure it connects to **R** (and Python if you have that installed). Click on “Start Session” in the top right. You should be able to click on the **R** version you have installed on your computer. If you have Python installed, you can easily switch back and forth between **R** and Python.

Section 2

Quarto

Reproducible Reports

The final product of a data analysis project is often a report: scientific publications, news articles, an analysis report for your company, or lecture notes for a class.

Now imagine after you are done you realize you:

- had the wrong data set
- have a new data set for the same analysis
- made a mistake and fix the error, or
- your boss or someone you are training wants to see the code and be able to reproduce the results

Situations like these are common for a data scientist.

Quarto

Quarto is a format for **literate programming** documents.

It is based on **markdown**, a markup language that is widely used to generate html pages.

You can learn more about markdown here:

<https://www.markdowntutorial.com/>

Quarto is similar to environments like Jupyter Notebooks, Google Colab, and R Markdown (R Markdown is very much like Quarto, but is dependent on **R**). Quarto is language agnostic. This means it natively works in both **R** and Python (and Julia and Observable JavaScript).

Quarto comes bundled with RStudio and Positron, but you could also download Quarto here: <https://quarto.org/docs/download/>.

Quarto Setup

Literate programming weaves instructions, documentation, and detailed comments in between machine executable code.

With Quarto, you need to **compile** the document into the final report. This allows the code to run in a clean workspace each time, so it is reproducible and gives the same results each time.

You can start a Quarto document in RStudio or Positron by clicking on **New** (then New File... in Positron) then **Quarto Document**. In RStudio, you will then be asked for a title and author.

Final reports can be output as to be HTML, PDF, Microsoft Word files or presentation formats like PowerPoint, Beamer, or HTML slides.

Quarto Setup

New Quarto Document

 Document

 Presentation

 Interactive

Title:

Author:

☒ HTML

Recommended format for authoring (you can switch to PDF or Word output anytime)

☐ PDF

PDF output requires a LaTeX installation (e.g. <https://yihui.org/tinytex/>)

☐ Word

Previewing Word documents requires an installation of MS Word (or Libre/Open Office on Linux)

Engine:

Editor: ☒ Use visual markdown editor [?](#)

[? Learn more about Quarto](#)

Create Empty Document

Create

Cancel

15

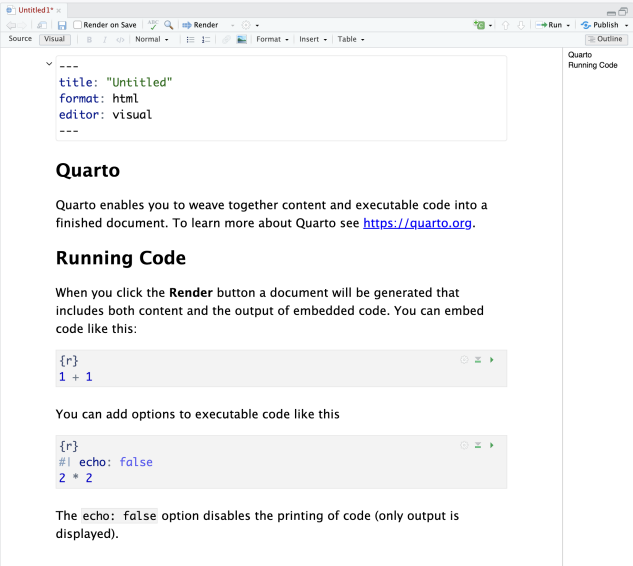
Quarto Setup

As a convention, we use the `.qmd` extension for these files.

In the template, you will see several things to note (in the following slides).

First, the default when opening a Quarto document is that it is in **Visual** editing mode. This will likely be a little more comfortable for you if you are not used to coding in something like LaTeX.

Quarto Visual



The screenshot shows the Quarto Visual editor interface. The top toolbar includes buttons for 'Render on Save', 'Render', 'Run', and 'Publish'. The 'Visual' tab is selected, showing a document with a title 'Untitled', a format of 'html', and an editor of 'visual'. The document content includes a section header 'Quarto', a paragraph about Quarto, another section header 'Running Code', a paragraph about the 'Render' button, and two code blocks. The first code block contains the expression $1 + 1$, and the second code block contains the expression $2 * 2$ with the option `#| echo: false` disabled. The right sidebar shows 'Quarto Running Code'.

```
---  
title: "Untitled"  
format: html  
editor: visual  
---
```

Quarto

Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see <https://quarto.org>.

Running Code

When you click the **Render** button a document will be generated that includes both content and the output of embedded code. You can embed code like this:

```
{r}  
1 + 1
```

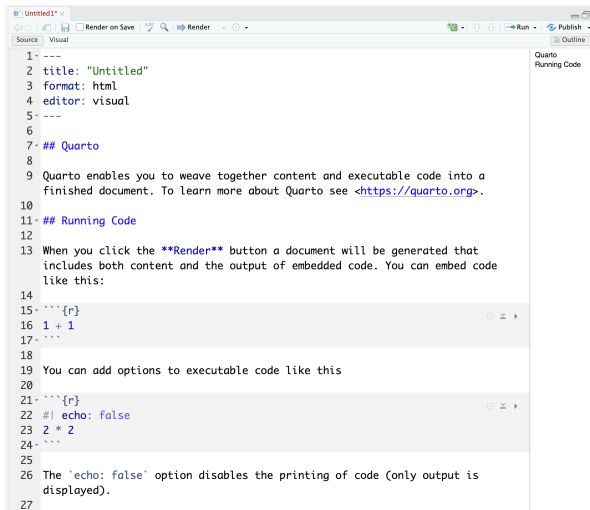
You can add options to executable code like this

```
{r}  
#| echo: false  
2 * 2
```

The `echo: false` option disables the printing of code (only output is displayed).

Quarto Source

However, we will focus on the source editor since it is much more customizable (but feel free to use the visual editor).



```
1 ---
2 title: "Untitled"
3 format: html
4 editor: visual
5 ---
6
7 ## Quarto
8
9 Quarto enables you to weave together content and executable code into a
10 finished document. To learn more about Quarto see <https://quarto.org>.
11
12 ## Running Code
13
14 When you click the Render button a document will be generated that
15 includes both content and the output of embedded code. You can embed code
16 like this:
17
18 ```{r}
19 1 + 1
20 ```
21
22 You can add options to executable code like this
23
24 ```{r}
25 #| echo: false
26 2 * 2
27 ```
28
29 The `echo: false` option disables the printing of code (only output is
30 displayed).
```

The Header

At the top of either the visual or source editor, you will see the following:

```
---  
title: "Untitled"  
format: html  
editor: visual  
---
```

This is known as the YAML header. YAML stands for YAML Ain't Markup Language.

One parameter that we will highlight is `format`. By changing this to, say, `pdf` or `docx`, we can control the type of output that is produced.

The Header

There are many, many options that can be adjusted in the YAML.

```
---  
title: "Project Title"  
author: "Your Name"  
date: "1/1/2026"  
format:  
  html:  
    code_folding: hide  
    toc: true  
    theme: "flatly"  
editor: source # Makes the source editor the default  
              # when file is opened.  
# Like R, you can put comments in the YAML.  
---
```

R Code Chunks

In various places in the document, we see something like this:

```
```${r}  
1 + 1
```
```

These are the code chunks. When you compile the document, the **R** code inside the chunk, in this case $1+1$, will be evaluated and the result included in that position in the final document.

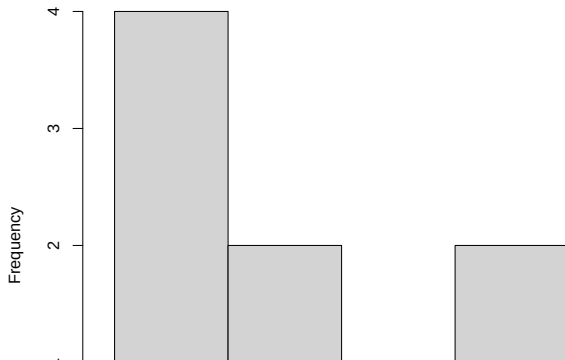
To add your own **R** chunks, you can type the characters above quickly with the key binding Command-Option-I (that is a capital i) on the Mac and Ctrl-Alt-I on Windows. You can also click on the green code chunk button in the toolbar at the top.

R Code Chunks

These code chunks can be used to show code, but that code will also run. Even plots can be made, although this plot is too big to fit on this slide. We will see soon how to adjust the size of plots.

```
x <- c(10, 15, 2, 4, 10, 5, 3, 5, 2)
hist(x)
```

Histogram of x



R Code Chunks

By default, both the code and the output will show up in your document. To avoid having the code show up, you can use an argument. To avoid this, you can set options for that code chunk using a special comment `#|` in the chunk. For example:

```
```{r}
#| echo: false
x <- c(10, 15, 2, 4, 10, 5, 3, 5, 2)
hist(x)
```
```

These comments need to be put at the beginning of the code chunk. Notice that this is YAML, not **R** code. That is why `false` is lowercase while in **R** it would be all capitalized.

R Code Chunks

Its a good habit to add a label to the **R** code chunks. This can be useful when debugging, among other situations. You do this by using `#| label:`

```
```{r}
#| label: x_hist
#| echo: false
x <- c(10, 15, 2, 4, 10, 5, 3, 5, 2)
hist(x)
```
```


Global Options and knitr

If there are certain options you want to apply to all code chunks, you can specify them in the YAML header with `execute:`. Be aware that white space matters in the YAML. The `echo` and `eval` options need to be indented, need to have a colon, and need to have one space after that colon.

```
---
title: "Untitled"
format: html
execute:
  echo: false
  eval: true
---
```

We use the `knitr` package to compile Quarto documents based on **R**, so we need to install that package. The specific function used to compile is the `knit()` function, which takes a filename as input. RStudio provides a Render button and Positron has a Preview button that does that for you, though.

Adding Equations

You can add formatted equations using between dollar signs \$. Example, if you type

```
$x^2+\frac{1}{2}$
```

It will look like $x^2 + \frac{1}{2}$. In fact, nearly all LaTeX code works in Quarto.

If you put two dollar signs, the equation will appear on its own line.

```
$$
```

```
\sum_{i=1}^{10}\left(\lambda_i+\frac{10}{\alpha}\right)
```

```
$$
```

$$\sum_{i=1}^{10} \left(\lambda_i + \frac{10}{\alpha} \right)$$

Adding and Adjusting Images

We can add images to our Quarto file by using

```
![image_caption_goes_here](image_name.ext){width=50%}
```

if the image is already in the same directory. Otherwise, you can put the path to the file in those parentheses. We can adjust the width of the image as well by changing the number in the `width=50%`.

Adding and Adjusting Images

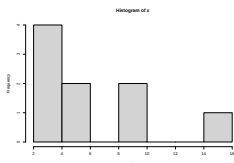
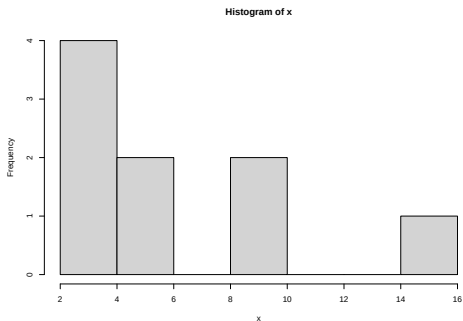
If the image is generated by **R** code, we can adjust the size by using `fig-width`, `fig-height`, `out-width` and/or `out-height` options in the code chunk.

```
```{r}
#| label: image_code
#| eval: true
#| echo: true
#| out-width: "25%"
x <- c(10, 15, 2, 4, 10, 5, 3, 5, 2)
hist(x)
```
```

The result is shown othe next slide.

Adding and Adjusting Images

The first image has out-width: "50%" and
the second has out-width: "25%"



Caching Code Chunks

Sometimes run chunks can take a while to run, so it makes sense to cache them. You can do that with `#| cache: true` so that the code chunk is only run once (and after each time it is edited). This will add some folders to your directory that Quarto uses to fetch needed information.

Be careful with caching! If a future code chunk that relies on the output of the cached code chunk is edited, Quarto will not re-run the cached code chunk. To make sure everything runs again, delete the folders created when caching.

Python in Quarto

One useful feature in Quarto is the ability to run code chunks from other languages. One of the most useful is the ability to run Python inside of Quarto.

There are two ways to do this:

- 1 Run Python natively.
- 2 Run Python through **R**.

Run Python in Quarto Natively

If you have Python and Jupyter installed, you can put `engine: jupyter` or `engine: jupyter3` in the YAML. Then you can put Python code chunks like this:

```
```{python}
import numpy as np
np.array([[1, 2, 3], [4, 5, 6]])
```
```

Note that **R** code chunks won't work when using the `jupyter` engine unless other options are specified.

Run Python in Quarto through **R**

We can run Python in Quarto through **R** using the `reticulate` package. This will make it a bit slower than running Python natively.

```
install.packages("reticulate")  
library(reticulate)
```

Installing Python and Python Packages Through R

To install Python, we can run the `install_python()` function from the `reticulate` package. Just indicate which Python version you want to install. As of the time these slides were created, the latest version is 3.14.5, but you can look up the latest version or go with the default.

```
install_python("3.14.5")
```

This can take a while to install. Please be patient!

To install Python packages, we need to use an **R** code chunk using the `py_install()` function.

```
py_install(packages = "matplotlib")  
py_install(packages = "numpy")  
py_install(packages = "pandas")
```

Running Python

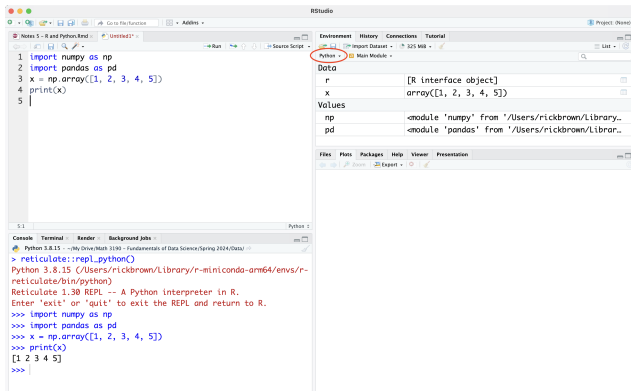
Once we have installed Python, we can insert a Python code chunk just like an **R** code chunk:

```
```{python}
import numpy as np
import pandas as pd
x_python = np.array([1, 2, 3, 4, 5])
print(x_python)
```
```

```
[1 2 3 4 5]
```

Python Envirnoment

When running Python in RStudio, the arrows in the console change and the environment switched to a Python environment. We can view what objects are stored in the **R** and Python environments by clicking on the drop down button.



Switch Between Python and **R** in RStudio

We can switch from an **R** environment (indicated by a single `>`) to a Python one (indicated by three arrows `>>>`) by

- Running a Python code chunk.
- Running the line `reticulate::repl_python()`.
- Double clicking on a Python object in the Python environment list.
- Clicking on the **R** icon at the top left of the Console to bring up a menu and then select Python.

We can switch from a Python environment back to **R** by

- Running an **R** code chunk.
- Typing `exit`.
- Clicking on the Python icon at the top left of the Console to bring up a menu and then select **R**.

Using Python Objects in **R** and Vice-Versa

We can send objects from Python to **R** and vice-versa. Everything defined in Python is stored in the `py` object, which can be called from **R** using `py$name_of_object`. To use an **R** object in Python, we can use preface it with `r.`, so it would be `r.name_of_object`.

```
# R Code. x_python was defined earlier.
```

```
library(reticulate)
```

```
py$x_python
```

```
[1] 1 2 3 4 5
```

```
y = 2 * py$x_python # Still R code
```

```
# Python Code
```

```
print(r.y)
```

```
[ 2.  4.  6.  8. 10.]
```

Type Conversions

While many objects are stored similarly in **R** and Python, it is good to know how things are stored when they are passed back and forth. This table shows the conversion:

| R | Python |
|------------------------|-------------------|
| Single-element vector | Scalar |
| Multi-element vector | List |
| List of multiple types | Tuple |
| Named list | Dict |
| Matrix/Array | NumPy ndarray |
| Data Frame | Pandas DataFrame |
| Function | Python function |
| Raw | Python bytearray |
| NULL, TRUE, FALSE | None, True, False |

Differences Between Quarto and R Markdown

If you've used R Markdown, much of what we discussed in Quarto will be familiar. In fact, most R Markdown code will run in Quarto unmodified. The main differences are that Quarto does not rely on **R**, chunk options are handled with special comments: `#|` instead of in the curly brackets, and some YAML options are a little different.

More info on differences can be found here:

<https://quarto.org/docs/faq/rmarkdown.html>.

In **R**, you can let the `knitr` package convert chunk options from R Markdown to Quarto style using

```
knitr::convert_chunk_header(input, output)
```

where `input` is the path to the file you want to convert and `output` is where you want the new file to be saved.

More on Quarto

There is a lot more you can do with Quarto. The documents that output are highly customizable. You can websites (seen later), blogs, books, Shiny apps, and more. There are many free resources on the internet including:

- 1 Posit's tutorial: <https://quarto.org>
- 2 The Quarto guide: <https://quarto.org/docs/guide/>
- 3 The Quarto definitive guide: <https://bookdown.org/yihui/rmarkdown>
- 4 The knitr book: <https://yihui.name/knitr/>
- 5 $\LaTeX$ command guide:
<https://www.bu.edu/math/files/2013/08/LongTeX1.pdf>

Extra - Tips

- Make sure you Render your document often. That way you will find any errors quickly without having to search for them.
- Comments in **R** or Python chunks still are the familiar `#`. But comments in Quarto documents outside of **R** chunks are `<!-- comment here -->`.
- In RStudio, you can bulk comment by highlighting all lines you want commented and then typing `Cmd-Shift-C` on a Mac or `Ctrl-Shift-C` on Windows.

Section 3

Introduction to Unix

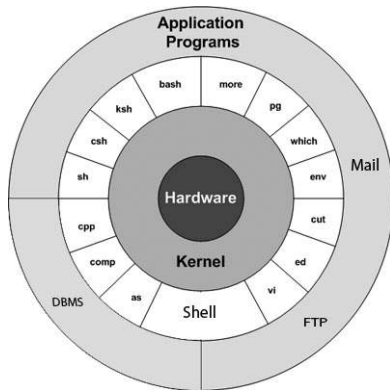
Unix Introduction

Unix is a family of multitasking, multiuser computer operating systems that derive from the original AT&T Unix. Originally developed in 1969 at Bell Labs. (Thanks Dennis Ritchie and Ken Thompson!)



Unix Introduction

The Unix operating system consists of many libraries and utilities along with the master control program, the kernel.



Features of Unix



History of Operating Systems

Most operating systems can be grouped into two lineages:

- Microsoft's Windows NT-based operating systems
- “Unix-like” operating systems (e.g., Linux, macOS, Android, iOS, Chrome OS, Orbis OS used on the PlayStation 4, firmware on your router)

UNIX

VS

Linux™




Darwin



Unix and Data Science

Unix is the operating system of choice in data science.

We will introduce you to the Unix way of thinking using an example: how to keep a data analysis project organized, and later how to do things more efficiently.

We will learn some of the most commonly used commands along the way.

Unix Resources

Here are some resources for more information:

- <https://www.codecademy.com/learn/learn-the-command-line>
- <https://www.edx.org/course/introduction-linux-linuxfoundationx-lfs101x-1>
- <https://www.coursera.org/learn/unix>

There are many reference books as well. Here are some particularly clear, succinct, and complete examples:

- <https://www.quora.com/Which-are-the-best-Unix-Linux-reference-books>
- <https://gumroad.com/l/bite-size-linux>
- <https://jvns.ca/blog/2018/08/05/new-zine--bite-size-command-line>

Naming Conventions

Before you start, pick a name convention that you will use to systematically name your files and directories. The Smithsonian Data Management Best Practices has “five precepts of file naming and organization”:

- Have a distinctive, human-readable name that gives an indication of the content.
- Follow a consistent pattern that is machine-friendly.
- Organize files into directories (when necessary) that follow a consistent pattern.
- Avoid repetition of semantic elements among file and directory names.
- Have a file extension that matches the file format (no changing extensions!)

The Tidyverse Style Guide¹ is highly recommended!

¹<https://style.tidyverse.org/>

Section 4

Unix Organization and Commands

Basic Unix Commands

In addition to clicking, dragging, and dropping files and folders, we will be typing Unix commands into the terminal. (Similar typing commands into the **R** console!)

You will need access to a terminal (see installation instructions in Installation Details section). Once you have a terminal open, you can start typing into the *command line*, usually after a \$ or % symbol. Once you hit enter, Unix will try to execute this command. For example:

```
echo "hello world"
```

Basic Unix Commands

The command `echo` is similar to `cat()` or `print()` in **R**. Executing this line should print out `hello world`, then return back to the command line.

Notice that you can't use the mouse to move around in the terminal like you ordinarily do. You have to use the keyboard or hold the Option key and then click on a Mac. To go back to a previous command, you can use the up arrow.

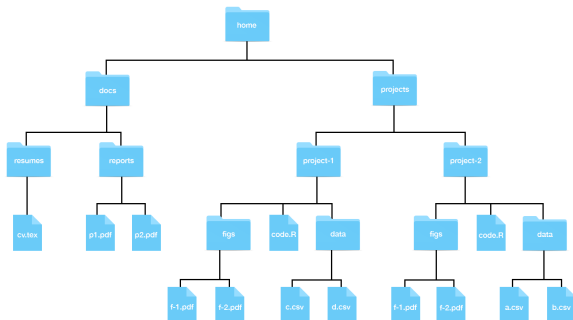
The Filesystem

We refer to all the files, folders, and programs on your computer as the **filesystem**. Keep in mind that folders and programs are also files (for later discussion). We will focus on files and folders for now and discuss programs, or **executables**, in a later section.

The first concept to understand is how your filesystem is organized; a series of nested folders, each containing files, folders, and executables.

Directories and Subdirectories

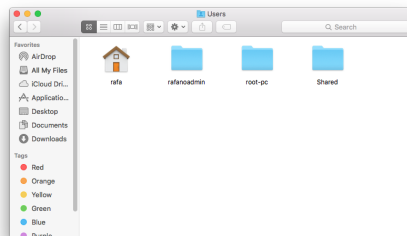
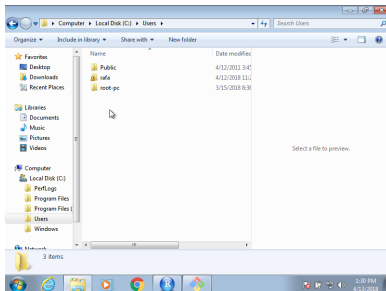
Here is a representation of the structure we are describing:



In Unix, we refer to folders as **directories**. Directories inside other directories referred to as **subdirectories**.

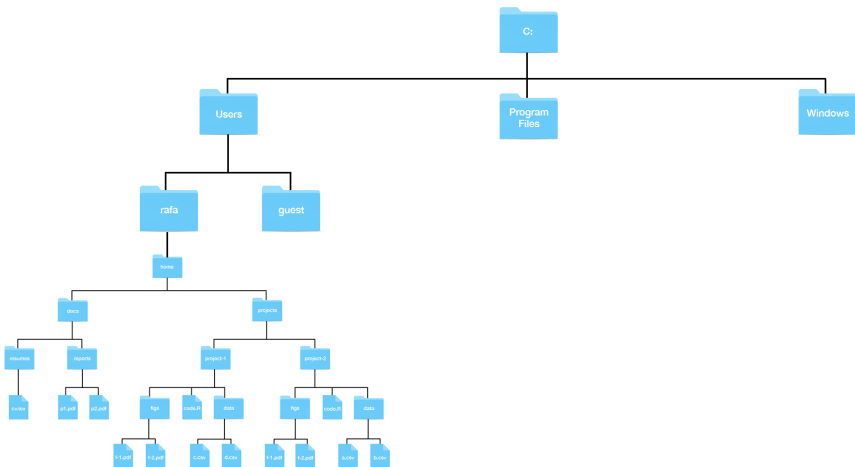
The Home Directory

The **home directory** is where your stuff is kept, as opposed to the system files, which are kept elsewhere. The name of your home directory is likely the same as your username. For example:



Windows Filesystem Structure

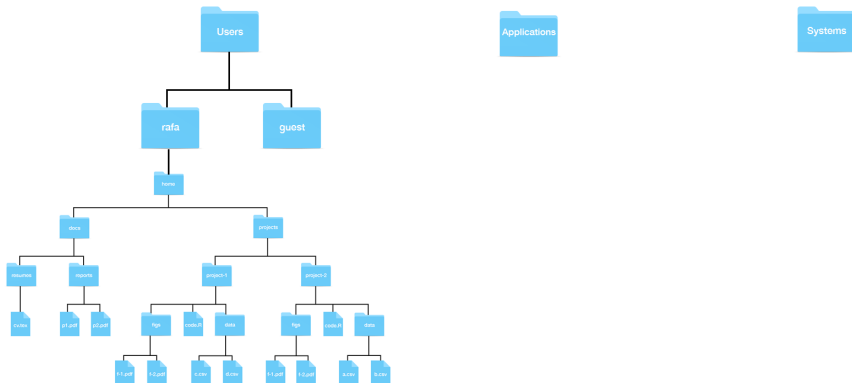
The Windows filesystem structure² looks like this:



²**Windows Users:** The typical **R** installation will make your Documents directory your home directory in **R**. This will likely be different from your home directory in Git Bash.

Mac Filesystem Structure

The Mac filesystem structure looks like this:



Working Directory

In Unix, the concept of a current location is indispensable. We refer to this as the **working directory**. Each terminal window you have open has a working directory.

How do we know our working directory? We use the Unix command: `pwd`, which stands for *print working directory*:

```
pwd
```

Paths

The string returned by `pwd` is the **full path** of the working directory. Every directory has a full path. We can also reference **relative paths**, which tell us how to get to a directory from the working directory.

In Unix, the shorthand `~` is a nickname for your home directory. So, for example, if `docs` is a directory in your home directory, the full path can be written `~/docs`.

Some Basic Unix Commands

To see the content of your home directory, open a terminal and type:

```
ls
```

You can also add **options** (`-` and `--`), **arguments**, and **wild cards** (`'*'`) to change function behavior:

```
ls -l
```

```
ls -a
```

```
ls -t
```

```
ls -r
```

```
ls -lart
```

```
ls -l *.txt
```

Some Basic Unix Commands

To create and remove directories, use the `mkdir` and `rmdir` functions, for example (use `ls` in between to see the directory come and go):

```
mkdir projects  
mkdir junk  
rmdir projects
```

Note: `rmdir` does not work if the directory is not empty.

Pro tip 1: The up arrow retrieves your previous commands.

Pro tip 2: You can auto-complete by hitting tab.

Pro tip 3: Type “Control-C” to clear a command.

Some Basic Unix Commands

To change the working directory use the `cd` command, which stands for *change directory*. For example:

```
cd projects
```

To check that it worked, use `pwd`. Now try:

```
cd .  
cd ..  
cd ../..  
cd /  
cd ~  
cd ~/projects  
cd ../junk
```

Some Basic Unix Commands

In Unix, we move files from one directory using the `mv` command:

```
mv path-to-file path-to-destination-directory
```

Warning: `mv` will not ask “are you sure?” if your move results in overwriting a file. Some `mv` examples:

```
mkdir ~/junk ~/projects/resumes
mv ~/projects/resumes/cv.tex ../../
mv ~/cv.tex ~/mycv.tex
mv ~/projects/mycv.tex junk/
mv ~/junk/mycv.tex ../projects/resumes/
```

Some Basic Unix Commands

The command `cp` behaves similar to `mv` except instead of moving, we copy the file. We first list the file we want to copy and then we list where we want to copy it and what its new name should be.

```
cp ~/projects/resumes/mycv.tex ~/mycv.tex
```

So in all the `mv` examples in the prior slide, you can switch `mv` to `cp` and they will copy instead of move. However to copy entire directories, add the recursive (`-r`) option:

```
cp -r ~/projects/resumes ~/junk/
```


Some Basic Unix Commands

In Unix, we can create a new blank file using the `touch` command.

```
touch filename
```

If that command is run again, then the file does not change except the modified date gets updated.

We remove files (and directories) with the `rm` command.

```
rm filename
```

Warning: Unlike throwing files into the trash or recycle bin, `rm` is permanent. Be careful! Note the following:

```
rm mycv.tex
```

```
rm junk/*.tex
```

```
rm -r projects junk
```

Some Basic Unix Commands

Unix uses an extreme version of abbreviations, which makes it hard to guess how to call commands. However, Unix includes complete help files or **man pages** (man is short for manual). In most systems, you can type `man` followed by the command name to get help. For example:

```
$ man ls
```

This command is not available in some of compact implementations of Unix (e.g., Git Bash). Alternatively, we can type the command followed by `-help`:

```
ls --help
```

Pro tip 4: Type `q` to quit out of a manual listing.

Advanced Unix functions

Most Unix implementations include a large number of powerful tools and utilities. It will take time to become comfortable with Unix, but as you struggle (if you do), you will find yourself learning just by looking up solutions on the internet.

Section 5

Git and GitHub

Git and GitHub Introduction

Many software developers and data scientists rely on Git and GitHub everyday.

There are three main reasons to use Git and GitHub.

- Sharing: GitHub allows us to easily share code with or without the advanced version control functionality.
- Collaborating: Multiple people make changes to code and keep versions synched. GitHub also has a special utility, called a **pull request**, that can be used by anybody to suggest changes to your code.
- Version control: The version control capabilities of Git permit us to keep track of changes, revert back to previous versions, and create **branches** in to test out ideas, then decide if we **merge** with the original.

See the Installation Details section for installing Git on your system.

GitHub Accounts

After installing Git, you need to create a GitHub account. To do this, go to <https://github.com> where you will see a link to sign up in the top right corner.

Pick a name carefully! Choose something short, somehow related to your name, and professional. Remember that you will use this to share code with others and you might be sharing this with potential collaborators or future employers!

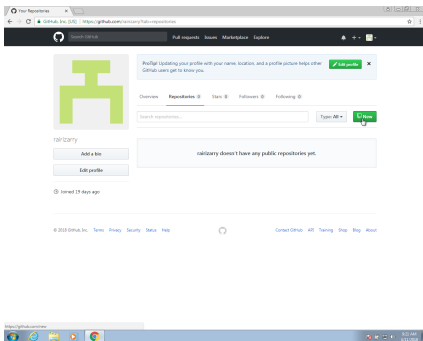
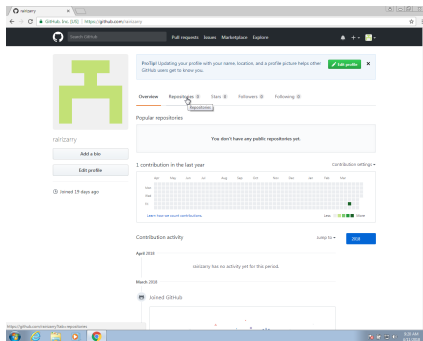
GitHub Repositories

A GitHub **repository** (repo for short) is like a folder on your computer and allows you to have two copies of your code: one on your computer and one on GitHub. If you add collaborators, then each will have a copy on their computer.

The GitHub copy is usually considered the **main** copy (previously called the **master** and still referred to as master in many tutorials and pages). Git will help you keep all the different copies synced.

GitHub Repositories

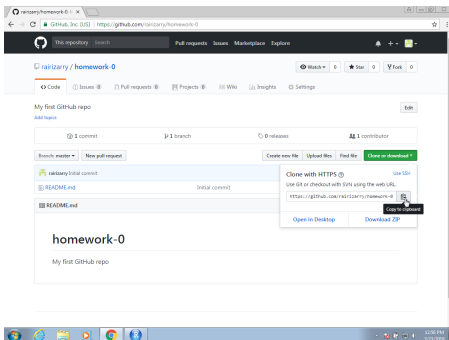
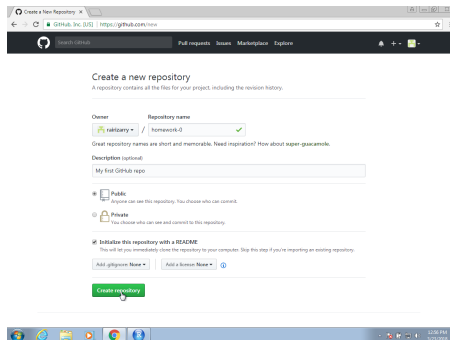
The first step is to initialize the repository on GitHub. You will have a page on GitHub with the URL: `http://github.com/username`. On your account, you can click on *Repositories* and then click on *New* to create a new repo:



GitHub Repositories

Choose a good descriptive name. In your first lab, you will create one called “Math_3190_Assignments”. You don't need to add a README.md, a .gitignore or choose a license at this time.

You will then be able to **clone** it on your computer using the terminal. Copy the link to connect to this repo for the next step.

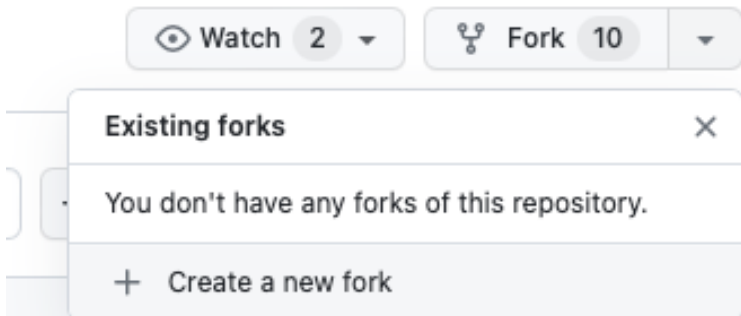


GitHub also allows you to **fork** others' repos. Go to <https://github.com/rbrown53/mypackage>

74

Forking

Click *Fork* in the top right and this will create a fork of the repository in your GitHub account. This will allow you to alter the code of other people (assuming you are allowed to do so by their license).



Git Setup - Tokens

Instead of using passwords, to connect local folders with GitHub repositories, we need to generate a GitHub **token**. Here are steps for obtaining one:

- ➊ Go to your GitHub home page: <https://github.com/username>
- ➋ Click on your account icon on the top right and go down to “Settings”.
- ➌ On the left side, at the bottom, click on “Developer Settings”.
- ➍ On the left, click on “Personal access tokens”.
- ➎ Click on Tokens (classic). We will be using the classic tokens, not the fine-grained tokens.
- ➏ Click on “Generate new token” and the “Generate new token (classic)”.
- ➐ Select an expatriation time. 90 days is good.
- ➑ Select “rep”, “workflow”, “gist”, and “user” under scopes.
- ➒ Click on “Generate token”.

More info can be found here:

<https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens>

Git Setup

Now we will work a bit in the Terminal. We need to let Git know who we are. This will make it clear who is making changes to files and directories. In a terminal window use the `git config` command:

```
git config --global user.name "My Name"  
git config --global user.mail "my@email.com"
```

Git Setup

The main actions in Git are to:

- 1 **pull** changes from the remote GitHub repo.
- 2 **add** files to the staging area, or as we say in the Git lingo: **stage** files.
- 3 **commit** changes to the local repository.
- 4 **push** changes to the **remote** GitHub repo.

Git Setup

To effectively permit version control and collaboration in Git, files move across four different areas:

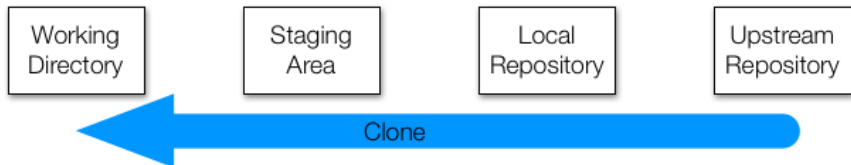


But how does it all get started? We can clone an existing repo or initialize one. We will explore cloning first.

Cloning Git Repositories

We will **clone** your existing my.package **upstream repository** to your local computer.

What does clone mean? We are going to actually copy the entire Git structure, files and directories to all stages: working directory, staging area, and local repository.



Cloning Git Repositories

Eventually, you will fork the mypackage repo from my GitHub. Then you will want to clone it to work on it on your local machine.

We can open a terminal and type:

```
cd ~  
pwd  
mkdir git-example  
cd git-example  
git clone https://github.com/yourusername/mypackage.git  
cd mypackage  
ls
```

Cloning Git Repositories

Note: the **working directory** is the same as your Unix working directory. When you edit files (e.g. in RStudio), you change the files in this directory. Git can tell you how these files relate to the upstream directory:

```
git status
```

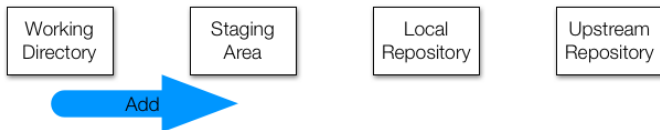


Now let's add some changes to the local repository. Open the DESCRIPTION file in the mypackage directory, and make a change.

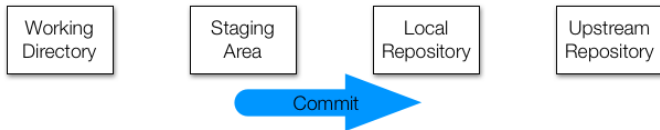
Working with Git Repositories

Now lets **add** the changes to the staging area and **commit** the changes to the local Git directory:

```
git add DESCRIPTION
```



```
git commit -m "updated DESCRIPTION with 'hello' function"
```



Working with Git Repositories

Now we can **push** the changes to the remote repo:

```
git push
```

The system will ask for your username and password. The username is your GitHub username, but the password is your personal token that we generated earlier.



Working with Git Repositories

We can also **fetch** any changes on the remote repo (how do you think this is different from clone?):

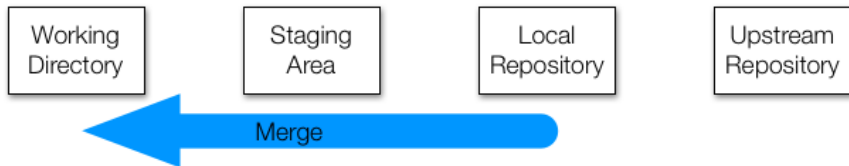
```
git fetch
```



Working with Git Repositories

And then we need to **merge** these changes to our staging and working areas:

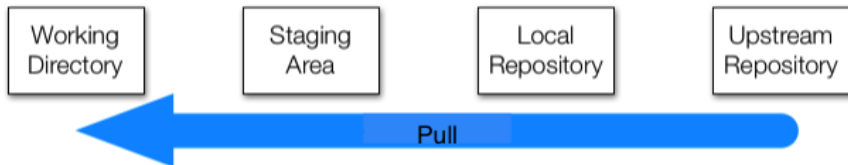
```
git merge
```



Working with Git Repositories

However, we often just want to change both with one command. For this, we use:

```
git pull
```



Important note: it is often a good idea to pull any changes when you start each day, so as to avoid **conflicts**.

Initializing a Git Directory

What if we already have a local directory and want to move it to a GitHub repository? See the following:

- 1 Create a new GitHub repository (e.g., Math_3190_Assignments)
- 2 **Initialize** the local repository

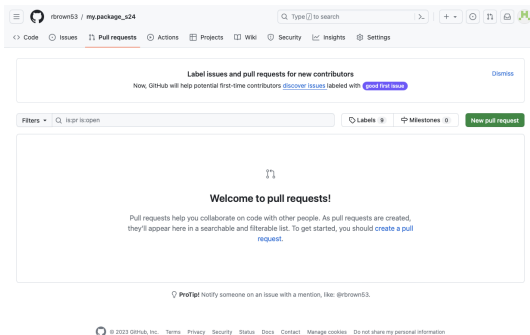
```
git init
```

- 3 Use the **add** and **commit** commands to add to the local repository
- 4 Connect the local and remote repos and push:

```
git remote add origin \  
https://github.com/username/Math_3190_Assignments.git  
git push -u origin main
```


Pull Requests

Pull requests enable sharing of changes from other branches/forks of a repo. Potential changes can be reviewed before they are merged into the base branch. More details on how to do these will be provided in the lab.



Extra - .gitignore File

When we initialize Git in a folder, a hidden folder called `.git` is created. Another hidden file that is useful is `.gitignore`, which lets Git know what files it should NOT keep track of. We can add this when creating a repo on GitHub or we can add this on the local machine by navigating to the directory in the terminal and typing

```
touch .gitignore
```

Extra - Hidden Files

These file and folders that begin with a period (.) are hidden by default, but sometime we want to edit them, especially the `.gitignore`. We can see hidden files by doing the following:

Mac: In Finder, type `Shift-Command-.` (period). Type that again to hide the files again.

Windows 10:

- 1 Open File Explorer from the taskbar.
- 2 Select `View > Options > Change folder and search options`.
- 3 Select the View tab and, in Advanced settings, select Show hidden files, folders, and drives and OK.

Windows 11:

- 1 Open File Explorer from the taskbar.
- 2 Select `View > Show > Hidden items`.

Conclusion

Positron also has a more visual version control that uses Git built in. It is on the left side and its symbol looks like a branching path.

On GitHub, you'll find a file called "Unix and Git Cheat Sheet.pdf" that contains all of the Git and Unix commands that are contained in these notes plus more.

Acknowledgements

These slides were adapted from slides created by Dr. W. Evan Johnson and some of the Git images are from *Introduction to Data Science: Data Analysis and Prediction Algorithms with R* by Rafael A. Irizarry.

Session info

```
sessionInfo()
```

```
R version 4.6.0 (2026-04-24)  
Platform: aarch64-apple-darwin23  
Running under: macOS Tahoe 26.4.1
```

```
Matrix products: default
```

```
BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBLAS.dylib  
LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib; LAPACK version 3.12.1
```

```
locale:
```

```
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: America/Denver
```

```
tzcode source: internal
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

```
other attached packages:
```

```
[1] reticulate_1.46.0
```

```
loaded via a namespace (and not attached):
```

```
[1] digest_0.6.39    fastmap_1.2.0    xfun_0.57        Matrix_1.7-5  
[5] lattice_0.22-9   knitr_1.51       htmltools_0.5.9  png_0.1-9  
[9] rmarkdown_2.31   tinytex_0.59     cli_3.6.6        grid_4.6.0  
[13] withr_3.0.2      compiler_4.6.0   rprojroot_2.1.1  here_1.0.2  
[17] rstudioapi_0.18.0 tools_4.6.0      evaluate_1.0.5   Rcpp_1.1.1-1.1  
[21] yaml_2.3.12      otel_0.2.0       rlang_1.2.0      jsonlite_2.0.0
```